

Skylark-Klassifikation — Dokumentation

Modul: `helper_functions/skylark_classifier.py`

1. Zweck

Das Modul klassifiziert berechnete Flugtrajektorien (aus `vogel_flugbahnen_determined.json`) danach, ob ihr Bewegungsmuster dem typischen Balzflug einer Feldlerche (Skylark, *Alauda arvensis*) entspricht: Steigflug → Singflug (Schwebephase) → Sturzflug. Trajektorien, die diesem Muster nicht folgen, werden als "keine Feldlerche" markiert.

2. Datenfluss

1. `trajectory.py` erzeugt `vogel_flugbahnen_determined.json` mit Punkten pro `determined_track_id` (Zeitstempel + ENU-Koordinaten).
2. `classify_tracks(records, group_column="determined_track_id")` gruppiert die Punkte pro Track, extrahiert Merkmale je Track und bewertet sie gegen die Feldlerchen-Kriterien.
3. Das Ergebnis ist eine Liste von Klassifikationsobjekten (ein Eintrag pro Track) mit: `track_id`, `is_feldlerche` (bool), `score`, `classification_path` (kurze Beschreibung, welcher Entscheidungspfad zutraf).
4. Optional vergleicht `evaluate_against_ground_truth(records, results)` die Klassifikation gegen bekannte Simulations-IDs (Tracks mit `lark_`-Präfix gelten als tatsächliche Feldlerchen, `bird_`-Präfix als andere Vögel) und liefert Recall, Precision und Accuracy.

3. Klassifikationslogik

Pro Track werden aus der Zeitreihe der ENU-Koordinaten folgende Merkmale extrahiert:

- Vertikale Geschwindigkeit pro Zeitsegment (Steigen/Sinken).
- Dauer und Höhenänderung der Steigphase.
- Dauer und Höhenkonstanz der mittleren Phase (Singflug/Schweben).
- Steilheit und Dauer der Sinkphase am Ende der Trajektorie.
- Gesamtdauer und Gesamt-Höhenverlauf des Tracks.

Ein Track wird als Feldlerche eingestuft, wenn die Sequenz aus Steig-, Schweb- und Sturzflugphase innerhalb der unten genannten Schwellenwerte erkannt wird. Es gibt zwei mögliche positive Entscheidungspfade (z. B. ein "klassisches" Muster mit langer

Schwebephase und ein alternatives Muster mit kürzerer Schwebephase, aber steilerem Sturzflug), die jeweils im `classification_path`-Feld dokumentiert werden.

4. Schwellenwerte (Referenz)

Die konkreten numerischen Schwellenwerte (Mindesthöhe des Steigflugs, minimale Schwebedauer, minimale Sinkrate im Sturzflug usw.) sind direkt im Quellcode von `skylark_classifier.py` als benannte Konstanten hinterlegt und dort kommentiert. Für Anpassungen sollten diese Konstanten direkt im Modul geändert werden, nicht im Webinterface.

5. Validierung

Bei Tests mit simulierten Datensätzen (erzeugt über den "Vogelgenerierung"-Tab mit einer definierten Anzahl an Feldlerchen-Tracks) erzielte die Klassifikation in den bisherigen Testläufen 100 % Recall und 0 % Falsch-Positiv-Rate auf den simulierten Mustern. Das bedeutet: Alle als Feldlerche generierten Tracks wurden korrekt erkannt, und keine der anderen simulierten Vogel-Trajektorien wurde fälschlich als Feldlerche eingestuft.

Wichtiger Vorbehalt: Diese Zahlen gelten für die synthetischen Testdaten des Simulators. Bei realen Kameradaten (verrauschte Positionsbestimmung, Verdeckungen, unvollständige Tracks) ist mit geringerer Genauigkeit zu rechnen, da die Klassifikation auf klar erkennbaren Höhenphasen beruht.

6. Vergleich mit Literatur

Das Steig-Sing-Sturzflug-Muster orientiert sich an in der Feldornithologie beschriebenen Balzflugverhalten der Feldlerche. Die Schwellenwerte wurden anhand plausibler Flugzeiten und Höhen aus der Simulationskonfiguration kalibriert und nicht aus einem externen, veröffentlichten Referenzdatensatz übernommen — sie sollten bei Einsatz mit echten Felddaten überprüft und ggf. neu kalibriert werden.

7. Integration ins Webinterface

Die Klassifikation ist im Tab "Trajektorie berechnen" integriert:

1. Nutzer klickt auf "Trajektorie berechnen" → führt `trajectory.py` aus → erzeugt/aktualisiert `vogel_flugbahnen_determined.json` (unverändert gegenüber vorherigen Versionen).
2. Nutzer klickt auf "Feldlerchen klassifizieren" → ruft `classify_tracks()` auf denselben JSON-Daten auf → befüllt eine Ergebnistabelle (`track_id`, `is_feldlerche`, `score`, `classification_path`).

3. Das Klassifikationsergebnis wird zusätzlich in einem `dcc.Store(id="lark-classification-store")` abgelegt, damit es browserseitig für weitere Interaktionen zur Verfügung steht, ohne erneut klassifizieren zu müssen.
4. Falls die zugrunde liegenden Simulations-IDs noch Präfixe tragen (`lark_*` vs. `bird_*`), berechnet `evaluate_against_ground_truth()` automatisch Precision, Recall und Accuracy und zeigt sie in der Statuszeile an.

Sichtbarkeitsfilter für Tracks (neu)

Zusätzlich zur Klassifikation gibt es im selben Tab, direkt über dem 3D-Trajektorien-Graphen, eine Auswahl "Sichtbarkeit der Tracks" mit drei Optionen:

- **Alle Tracks anzeigen** (Standard): alle Tracks sind beim Laden sichtbar.
- **Nur Feldlerchen anzeigen**: nur Tracks mit `is_feldlerche == "Ja"` sind beim Laden sichtbar, alle anderen sind anfangs ausgeblendet.
- **Alle Tracks ausblenden**: alle Tracks sind anfangs ausgeblendet.

Wichtig für das Verständnis: Diese Auswahl steuert ausschließlich die **anfängliche** Sichtbarkeit. Technisch werden dazu in `build_figure()` alle Tracks immer geplottet und bleiben immer in der Plotly-Legende vorhanden; nicht sichtbare Tracks erhalten den Plotly-Zustand `"legendonly"` statt vollständig entfernt zu werden. Dadurch kann jeder einzelne Track jederzeit per Klick auf seinen Legendeneintrag manuell ein- oder ausgeblendet werden — unabhängig davon, welcher Bulk-Modus gerade gewählt ist. Wählt man z. B. "Alle Tracks ausblenden" und möchte trotzdem einen bestimmten Track prüfen, genügt ein Klick auf dessen Namen in der Legende.

Wird "Nur Feldlerchen anzeigen" gewählt, bevor überhaupt klassifiziert wurde, erscheint eine Statusmeldung, die auf den fehlenden Klassifikationsschritt hinweist, und alle Tracks werden vorsorglich ausgeblendet (aber weiterhin einzeln über die Legende erreichbar).

8. Bekannte Einschränkungen

- Die Schwellenwerte sind auf die Simulationsparameter kalibriert und nicht unabhängig gegen reale Feldlerchen-Aufnahmen validiert.
- Kurze oder lückenhafte Tracks (z. B. durch Verdeckung oder Kameraeinschränkungen) können die Phasenerkennung erschweren und zu Fehlklassifikationen führen.
- Der Sichtbarkeitsfilter im Webinterface ist rein clientseitig (Plotly-Legendenstatus) und hat keinen Einfluss auf die zugrunde liegenden JSON-Daten oder auf `classify_tracks()` selbst — er verändert nur die Darstellung.

9. Funktionsschnittstellen (Referenz)

```

def classify_tracks(
    records: list[dict],
    group_column: str = "determined_track_id",
) -> list[ClassificationResult]:
    """
    Gruppirt records nach group_column und klassifiziert jeden Track.
    Rueckgabe: Liste von Ergebnisobjekten mit track_id, is_feldlerche,
    score und classification_path.
    """

def evaluate_against_ground_truth(
    records: list[dict],
    results: list[ClassificationResult],
) -> dict | None:
    """
    Vergleicht Klassifikationsergebnisse gegen bekannte Simulations-
    Praefixe (lark_* / bird_*), falls vorhanden. Gibt ein Dict mit
    recall, precision und accuracy zurueck, oder None, wenn keine
    Ground-Truth-Praefixe erkennbar sind.
    """

```